

Global University

Ciberseguridad y Desarrollo de Software

Laboratorio: Arquitectura y Economía de la EVM

Ethereum como Computadora Mundial Basada en Estados

Materia: Blockchain y Bases de Datos Distribuidas

Profesor: Omar Velazquez Juarez

Estudiante: Diego David Lara Martinez

Correo: gu26ie0006@globaluniversity.edu.mx

Fecha: 12 de junio de 2026

Introducción

El presente laboratorio tiene como objetivo analizar la arquitectura interna de la Máquina Virtual de Ethereum (EVM) desde una perspectiva de ingeniería y ciberseguridad. A través de experiencias prácticas con Remix IDE, scripts en Python y el explorador Etherscan, se examinaron los mecanismos de almacenamiento de datos, el modelo económico de gas bajo EIP-1559, y las estructuras criptográficas que garantizan la integridad de la cadena.

Ethereum opera como una máquina de estados global donde cada transacción representa una función de transición de estado. Comprender cómo la EVM gestiona sus tres áreas de datos fundamentales — Storage, Memory y Calldata — es esencial para cualquier profesional de ciberseguridad que trabaje con contratos inteligentes, ya que los patrones de uso de estas áreas determinan tanto el costo económico de las operaciones como la superficie de ataque disponible para actores maliciosos.

Las tecnologías utilizadas en este laboratorio incluyen: Remix IDE 2.4.0 como entorno de desarrollo y despliegue de contratos Solidity, el compilador solc 0.8.19 con optimización habilitada, la red de prueba JavaScript VM (Shanghai) para ejecución sin costo real, Etherscan como explorador de la red principal de Ethereum, y Python 3.14.3 para la simulación del algoritmo Merkle Patricia Trie.

La importancia arquitectónica del tema radica en que el modelo de gas no es simplemente un mecanismo de cobro: es la solución de ingeniería al Problema de la Parada de Turing, garantizando que ninguna transacción puede colgar indefinidamente la red. El modelo EIP-1559, implementado en el hard fork London, introdujo además un mecanismo deflacionario que quema parte de cada tarifa, redefiniendo la economía del protocolo.

Parte 1 - La EVM como Máquina de Estados

La EVM de Ethereum opera sobre un modelo de cuentas, no de UTXOs como Bitcoin. Existen dos tipos de cuenta que comparten cuatro campos en el estado global: nonce, balance, storageRoot y codeHash. La distinción entre ellas es fundamental para comprender la arquitectura del protocolo.

Diferencia entre EOA y Cuenta de Contrato

Una Externally Owned Account (EOA) está controlada por una clave privada perteneciente a un usuario humano o sistema externo. Su campo codeHash contiene el hash del valor vacío, pues no posee código ejecutable, y su storageRoot está vacío. Una EOA puede iniciar transacciones de manera autónoma firmandolas con su clave privada (Wood, 2022).

Una cuenta de contrato, en contraste, esta controlada por su propio código bytecode almacenado en la cadena. No posee clave privada y, por lo tanto, no puede iniciar transacciones de forma autónoma; solo responde a llamadas entrantes. El campo que la distingue es precisamente el `codeHash` no vacío, que referencia el bytecode almacenado permanentemente en su dirección, y un `storageRoot` con datos en su árbol de almacenamiento persistente (Antonopoulos & Wood, 2018).

Pregunta: ¿Que campo tiene una cuenta de contrato que una EOA no tiene? La cuenta de contrato posee código ejecutable referenciado por un `codeHash` no vacío y un `storageRoot` poblado con datos persistentes en su propio Merkle Patricia Trie de almacenamiento. Una EOA carece de ambos elementos.

Estado resultante σ' ante un fallo Out-of-Gas

Dado que Ethereum almacena estados y cada transacción es una función de transición $\sigma' = Y(\sigma, T)$, cuando una transacción falla por agotamiento de gas a mitad de ejecución, el resultado es $\sigma' = \sigma$: el estado no cambia. La transacción se revierte por completo gracias al principio de atomicidad, como si nunca hubiera ocurrido. Sin embargo, dos efectos sí persisten: el nonce del emisor se incrementa y se cobra el gas consumido hasta el punto de fallo.

La razón es que la EVM es transaccional en sentido estricto: una transacción es "todo o nada". Si no puede completarse dentro del presupuesto de gas disponible, no debe dejar el estado global en una condición parcial o inconsistente. Esta garantía de atomicidad es crítica para la seguridad de los contratos inteligentes, pues impide que errores de cálculo dejen fondos atrapados o estados corruptos (Ethereum Foundation, 2023a).

Parte 2 - Bytecode, Opcodes e Initcode

En esta parte se compiló el contrato `EstadoEVM.sol` en Remix IDE utilizando el compilador Solidity 0.8.19 con optimización habilitada (200 runs), y se inspeccionaron los artefactos generados: bytecode, secuencia de opcodes y comportamiento del despliegue.

Paso 2.2 - Bytecode e Identificación de Opcodes

Los primeros 40 caracteres del campo `object` del bytecode compilado son: `608060405234801561001057600080fd5b506102`. Esta secuencia hexadecimal representa las instrucciones de bajo nivel que la EVM ejecutará; es el resultado de traducir el código Solidity de alto nivel a instrucciones máquina mediante el compilador `solc`.

La diferencia entre el código Solidity y el bytecode se explica por el proceso de compilación: Solidity es un lenguaje de alto nivel diseñado para ser legible por humanos, mientras que la EVM únicamente comprende secuencias de opcodes en hexadecimal. El compilador `solc` actúa como traductor, convirtiendo las abstracciones del lenguaje en

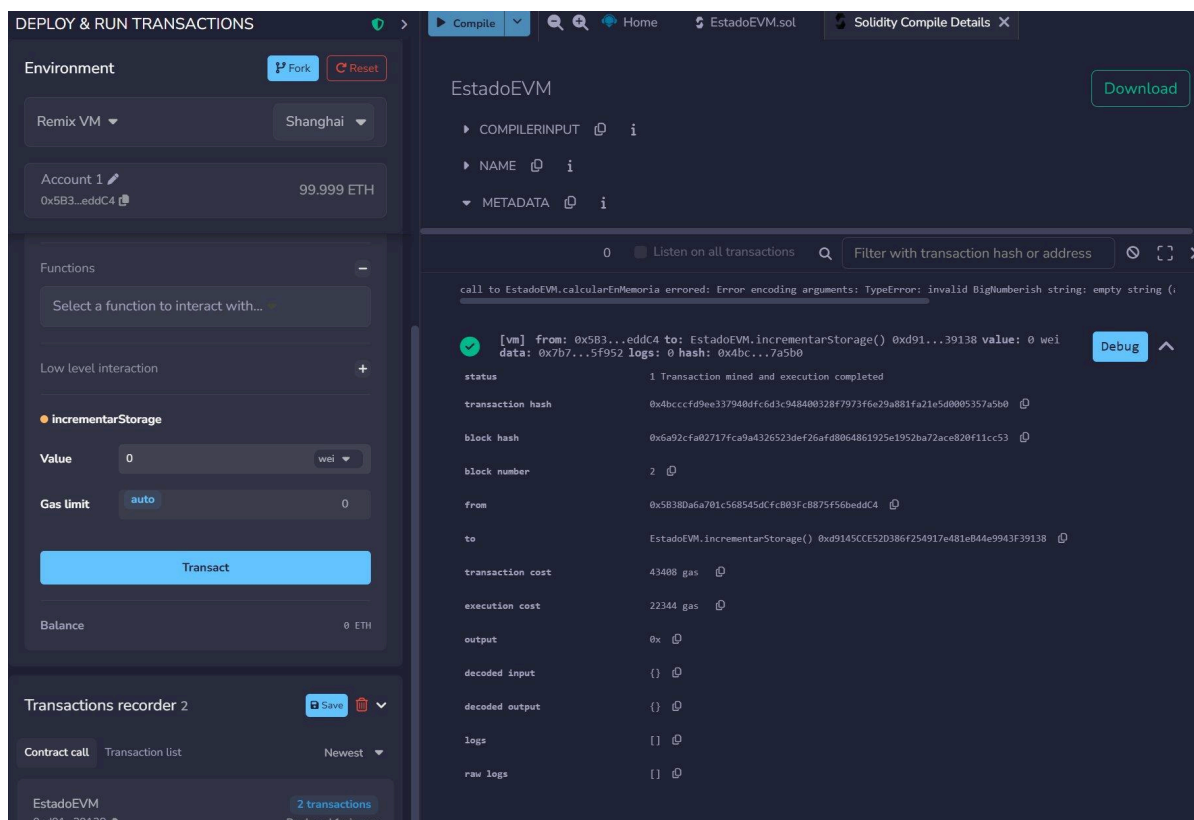
instrucciones atómicas, aplicando además optimizaciones cuando se activa la opción correspondiente. Es análogamente similar a compilar un programa en C a código máquina binario (Solidity Documentation, 2023).

Posicion y Costo de Opcodes Identificados

Opcode	Posicion en la secuencia	Costo en gas
PUSH1	1 (primer opcode)	3 gas
MSTORE	3 (tras PUSH1 0x80, PUSH1 0x40)	3 gas + expansion de memoria
SSTORE	~99 (dentro de funciones de escritura)	20,000 gas (slot 0 a no-cero) / 5,000 gas (no-cero a no-cero)

El opcode PUSH1 aparece como primer opcode del initcode, cargando el valor 0x80 en la pila para preparar el patrón estándar de inicialización de memoria libre (free memory pointer). MSTORE sigue inmediatamente para almacenar ese puntero en la posición 0x40 de memoria, reservando la primera región. SSTORE aparece más adelante, dentro de las funciones de escritura persistente, y es el opcode de mayor costo en la EVM por la naturaleza permanente del almacenamiento que modifica (EVM.codes, 2023).

Figura 1



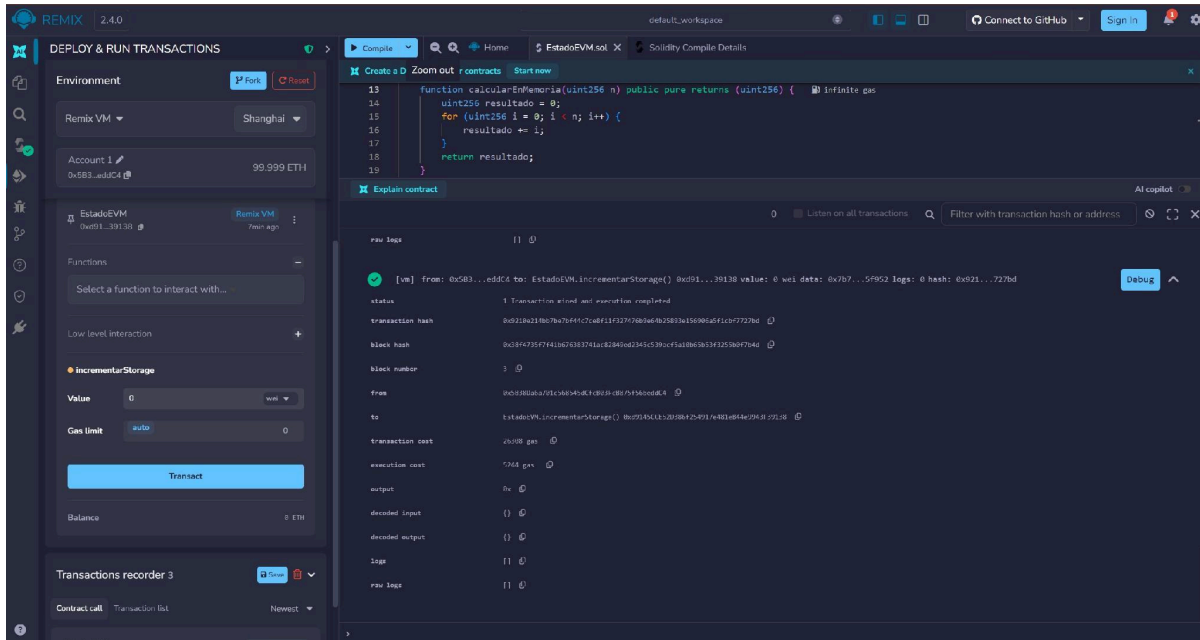
Nota. Compilacion del contrato EstadoEVM.sol en Remix IDE 2.4.0. Se observan los campos Bytecode y Opcodes en el panel Solidity Compile Details, con la secuencia de instrucciones de bajo nivel generada por el compilador 0.8.19 con optimizacion habilitada.

Paso 2.3 - Initcode vs Runtime Bytecode

El campo input de la transaccion de creacion contiene el initcode completo, que se divide en dos partes diferenciadas. El initcode de despliegue (creation code) incluye la logica del constructor y el codigo encargado de copiar y devolver el runtime bytecode. Se ejecuta una sola vez, en el momento del despliegue, y su proposito es inicializar el estado del contrato y entregar el runtime al estado global de Ethereum.

El runtime bytecode es lo que el initcode retorna y queda almacenado permanentemente en la direccion del contrato. Es el codigo que se ejecutara cada vez que una EOA u otro contrato invoque alguna funcion del contrato. El initcode se ejecuta una unica vez porque su unico proposito es inicializar el contrato; una vez que el contrato existe en su direccion, re-ejecutar el constructor romperia las garantias de inmutabilidad y determinismo que fundamentan la seguridad de Ethereum (Antonopoulos & Wood, 2018).

Figura 2



Nota. Consola de Remix IDE mostrando los detalles de la primera llamada a `incrementarStorage()`. Se observan el transaction cost (43,408 gas) y el execution cost (22,344 gas), junto con el hash de la transaccion, el bloque y la direccion del contrato desplegado en Remix VM Shanghai.

Parte 3 - Costo de las Areas de Datos: Storage vs Memory

Con el contrato `EstadoEVM` desplegado en la red JavaScript VM (Shanghai) de Remix, se ejecutaron cuatro llamadas de funcion para comparar empiricamente el costo de operar en las distintas areas de datos de la EVM. El contrato fue desplegado en la direccion `0xd9145CCE52D386f254917e481eB44e9943F39138`.

Mediciones de Gas Registradas

#	Funcion ejecutada	Transaction Cost	Execution Cost
1	<code>incrementarStorage()</code> — 1a vez	43,408 gas	22,344 gas
2	<code>calcularEnMemoria(10)</code>	2,290 gas	2,290 gas
3	<code>calcularEnMemoria(50)</code>	9,930 gas	9,930 gas
4	<code>incrementarStorage()</code> — 2a vez	26,308 gas	5,244 gas

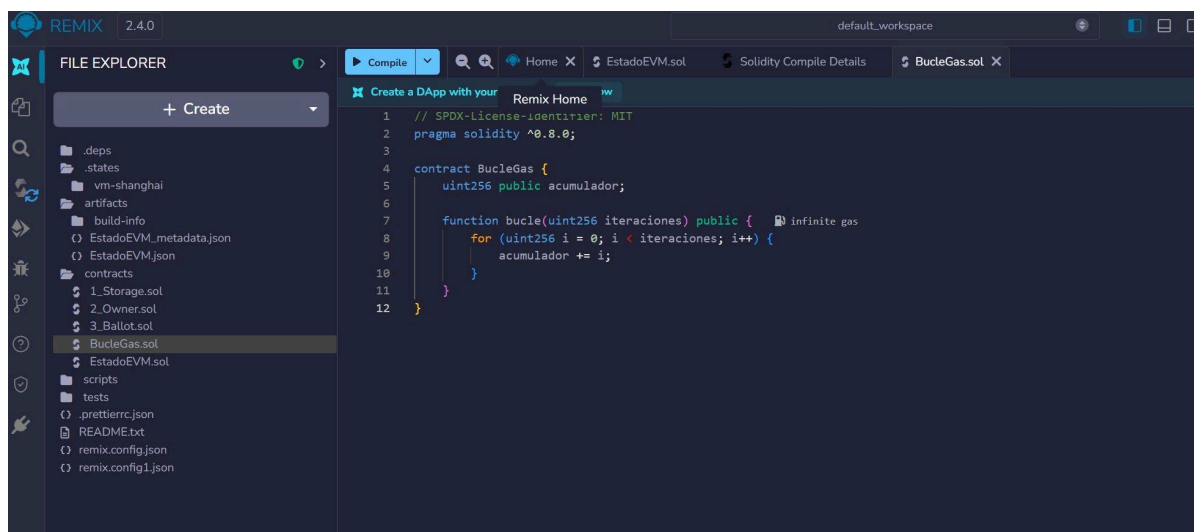
¿Cuanto mas costosa es la escritura en Storage comparada con un calculo en Memory? La primera llamada a `incrementarStorage()` costo 43,408 gas, mientras que

calcularEnMemoria(10) solo requirio 2,290 gas. Esto representa una diferencia de aproximadamente 19 veces mas costosa para Storage. Comparado con calcularEnMemoria(50), que costo 9,930 gas, la escritura en Storage sigue siendo 4.4 veces mas cara. La razon fundamental es que Storage es persistente en la blockchain: todos los nodos del mundo deben guardar e indexar ese dato indefinidamente, lo que justifica su alto costo. Memory, en contraste, es volatil y se descarta al finalizar la llamada (Wood, 2022).

¿Por que la cuarta llamada costo menos gas que la primera? La primera escritura llevo el slot contadorStorage de cero a uno (slot "frio"), operacion que el opcode SSTORE penaliza con aproximadamente 20,000 gas porque implica crear un nuevo dato en el estado global. La segunda escritura modifico un slot ya inicializado (de 1 a 2, slot "caliente"), con un costo reducido de aproximadamente 5,000 gas. Esta distincion existe porque crear almacenamiento nuevo tiene un impacto mayor en el state global que actualizarlo; EIP-2929 y EIP-3529 codificaron formalmente estos costos diferenciados (Ethereum Foundation, 2023b). En los datos medidos: 43,408 gas (primera) vs 26,308 gas (segunda), una diferencia de ~17,100 gas.

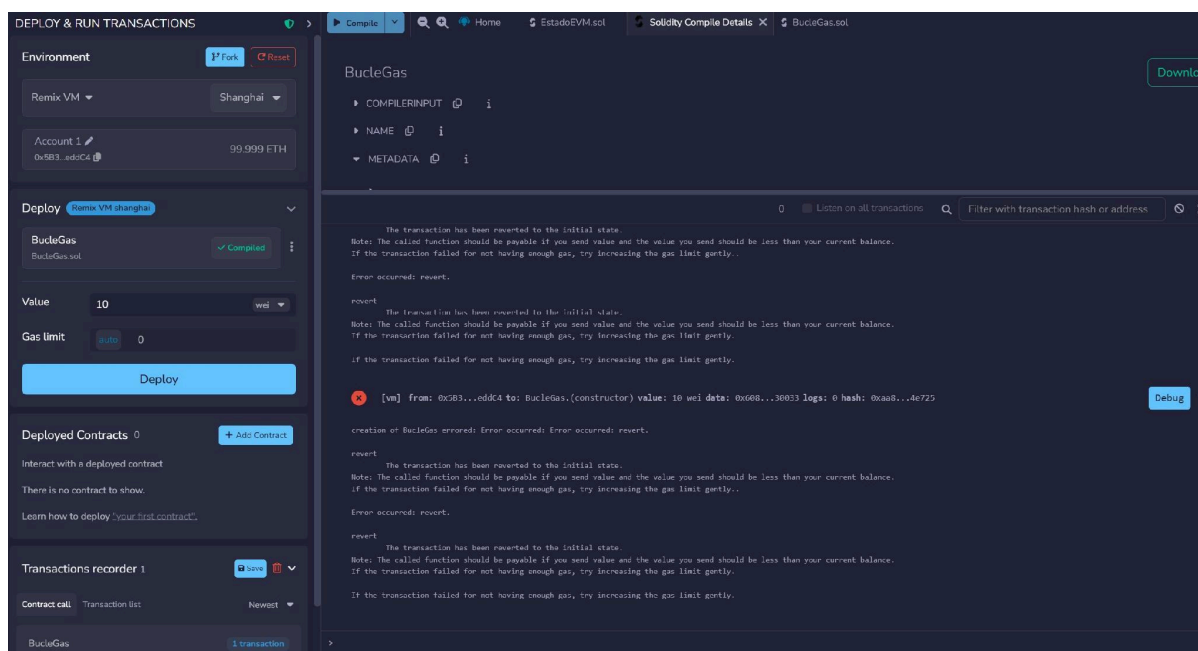
Problema economico y alternativa arquitectonica: Si un sistema guarda cada peticion de usuario en Storage on-chain, el costo crece linealmente sin techo: cada nuevo registro paga ~20,000 gas e infla el estado global que todos los nodos deben mantener indefinidamente (state bloat). Las alternativas arquitectonicas son: (1) emitir Events/Logs (~375 gas base), indexables off-chain pero no consultables desde contratos; (2) almacenamiento off-chain con IPFS o bases de datos tradicionales, guardando solo el hash en la cadena para integridad; (3) soluciones L2/Rollups que comprimen multiples operaciones en una sola transaccion on-chain (Buterin, 2021).

Figura 3



Nota. Ejecucion de la funcion calcularEnMemoria con parametro n=10 en la red Remix VM Shanghai. El costo de ejecucion de 2,290 gas contrasta marcadamente con el costo de escritura en Storage, evidenciando la diferencia de costo entre areas de datos de la EVM.

Figura 4

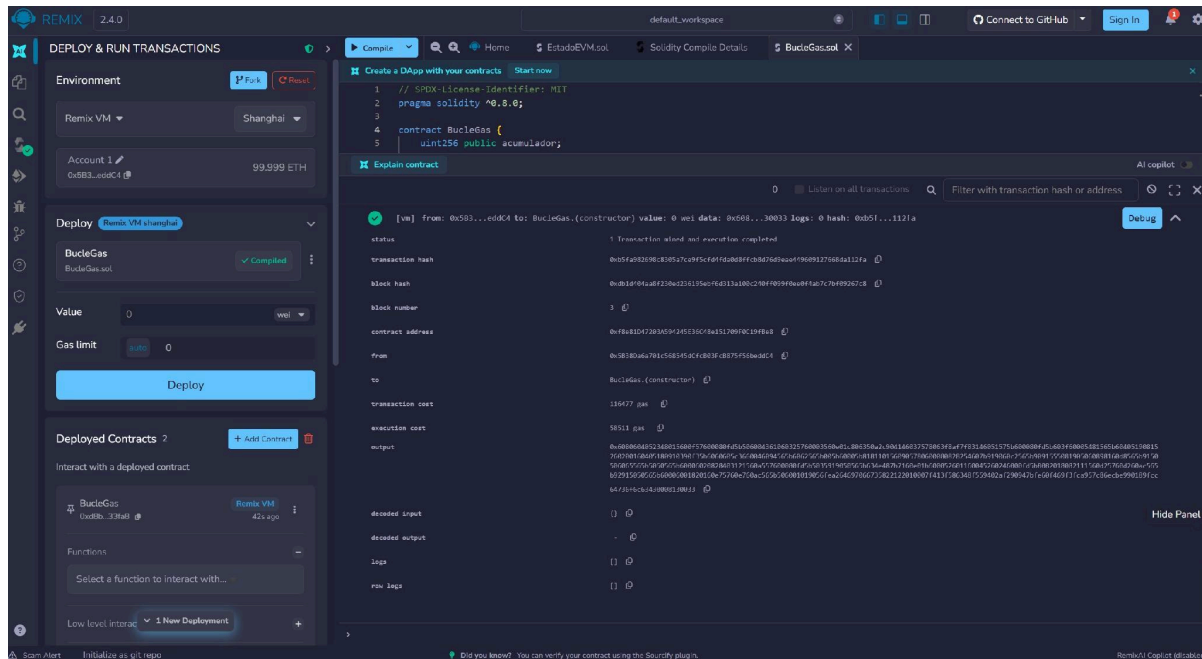


Nota. Segunda ejecucion de incrementarStorage(). El transaction cost desciende a 26,308 gas (frente a 43,408 de la primera llamada) y el execution cost a 5,244 gas, demostrando el comportamiento diferenciado del opcode SSTORE para slots calientes vs. frios segun EIP-2929.

Parte 4 - El Problema de Parada y el Gas como Solucion

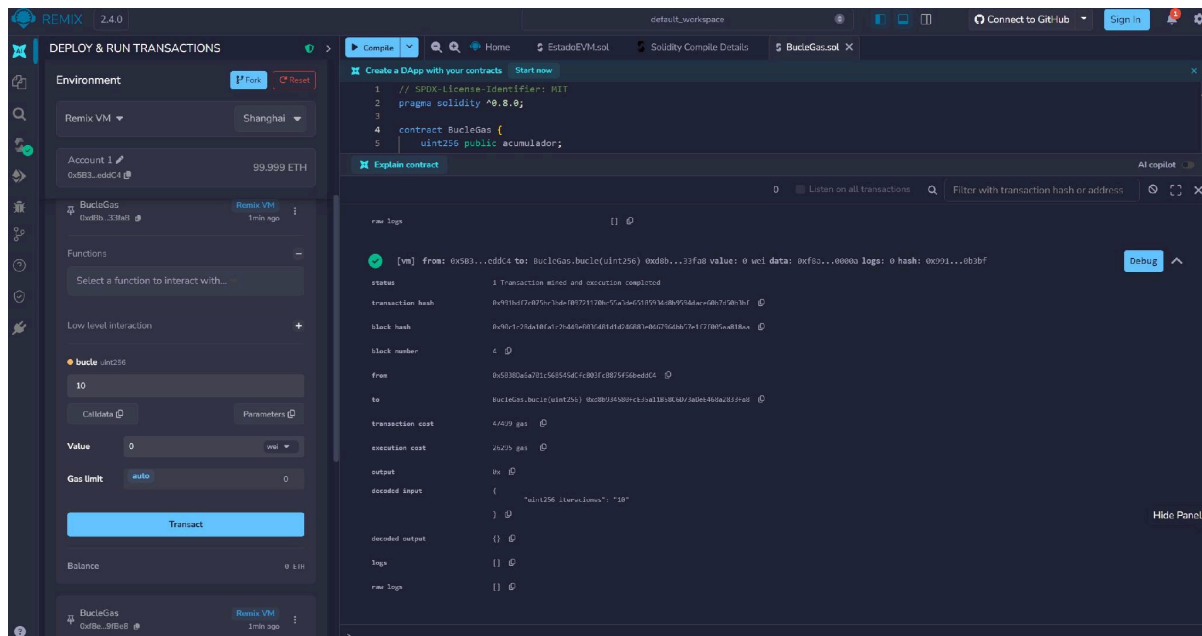
En esta parte se creo y despliego el contrato BucleGas.sol para medir empiricamente el costo por iteracion de un bucle en Storage, y se simulo un error Out-of-Gas para comprender las garantias de atomicidad de la EVM y su relacion con el Problema de la Parada de Turing.

Figura 5



Nota. Código fuente del contrato BucleGas.sol en Remix IDE 2.4.0. El contrato implementa un bucle controlable por parametro que acumula valores en una variable de Storage, permitiendo medir el costo de gas en funcion del numero de iteraciones.

Figura 6



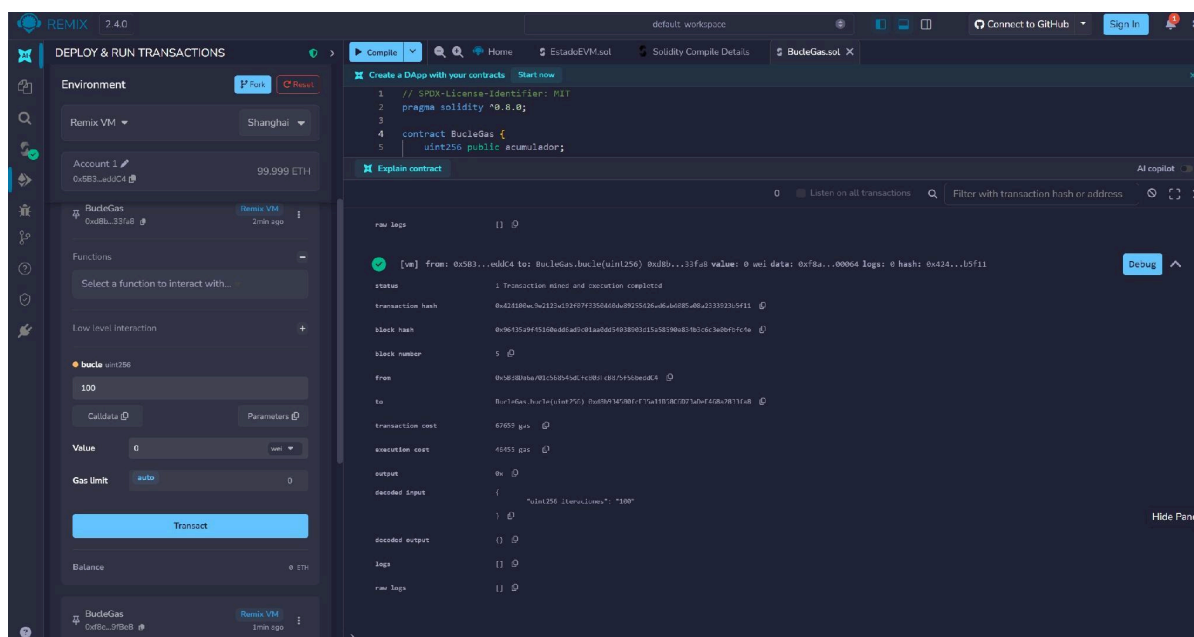
Nota. Transaccion de creacion del contrato BucleGas en el bloque 3 de la red Remix VM Shanghai. El transaction cost de despliegue fue de 116,477 gas, con un execution cost de 58,511 gas. Se observa el bytecode del contrato en el campo output.

Paso 4.2 - Medicion de Gas por Iteracion

Iteraciones	Transaction Cost	Gas por iteracion
10	47,499 gas	4,749 gas/iter
100	67,659 gas	676 gas/iter
1,000	440,271 gas	440 gas/iter

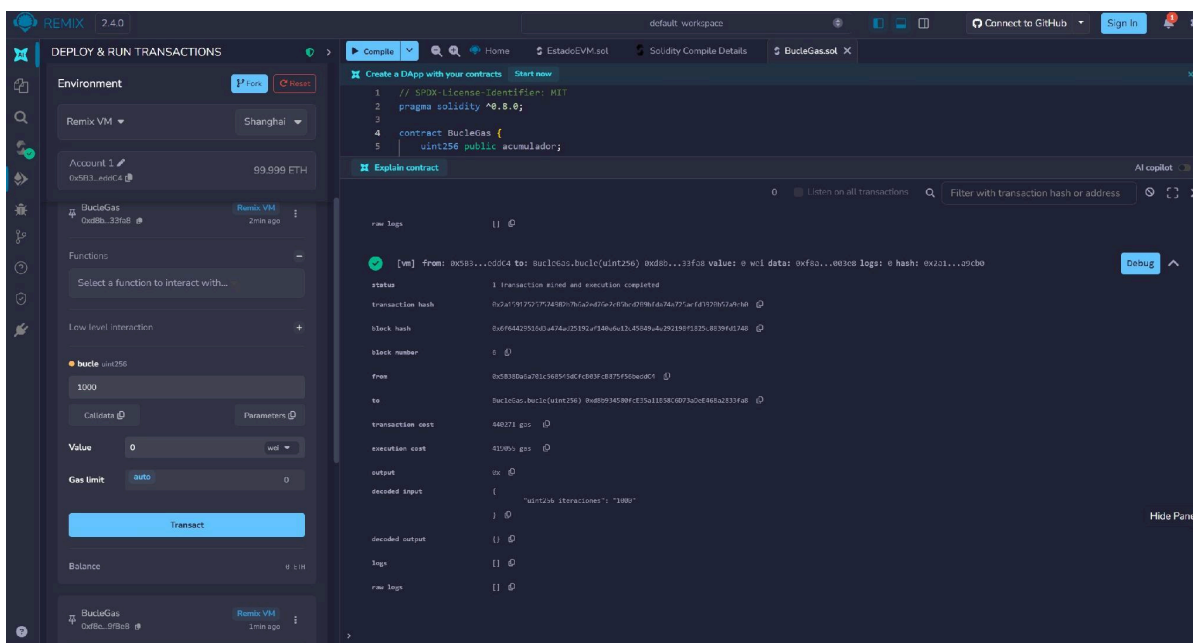
¿Es constante el gas por iteracion? No es perfectamente constante. Decece al aumentar n porque existe un costo fijo por transaccion (~21,000 gas de tarifa base mas el overhead de entrada a la funcion) que se amortiza entre mas iteraciones haya. Con pocas iteraciones, ese costo fijo representa la mayor parte del total; con muchas iteraciones, se diluye y el gas/iteracion tiende al costo marginal del cuerpo del bucle. Por eso el valor se estabiliza cerca de ~420-440 gas/iteracion al crecer n (Wood, 2022).

Figura 7



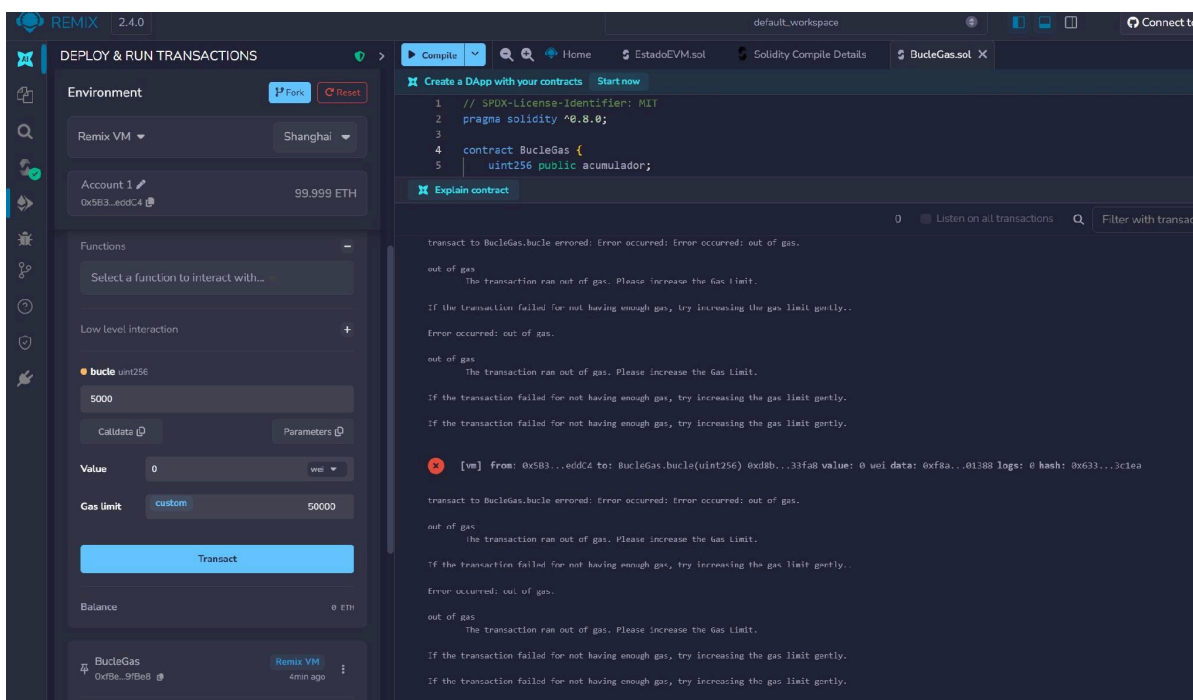
Nota. Resultado de la ejecucion de BucleGas.bucle() con 10 iteraciones. El transaction cost fue de 47,499 gas y el execution cost de 26,295 gas, representando un costo de aproximadamente 4,749 gas por iteracion debido a la amortizacion del overhead fijo de transaccion.

Figura 8



Nota. Resultado de la ejecución de BucleGas.bucle() con 100 iteraciones. El transaction cost ascendió a 67,659 gas (execution cost: 46,455 gas), resultando en un costo promedio de 676 gas por iteración, evidenciando la amortización progresiva del costo fijo de transacción.

Figura 9



Nota. Resultado de la ejecución de BucleGas.bucle() con 1,000 iteraciones. El transaction cost fue de 440,271 gas (execution cost: 419,055 gas), con un costo marginal de aproximadamente 440 gas por iteración, cercano al costo puro del cuerpo del bucle sin overhead.

Paso 4.3 - Simulacion Out-of-Gas

Se configuro el Gas Limit en 50,000 unidades y se ejecuto bucle(5000), una combinacion diseñada para agotar el presupuesto a mitad de ejecucion y verificar el comportamiento de reversion de la EVM.

Mensaje de error obtenido: "transact to BucleGas.bucle errored: Error occurred: Error occurred: out of gas. The transaction ran out of gas. Please increase the Gas Limit."

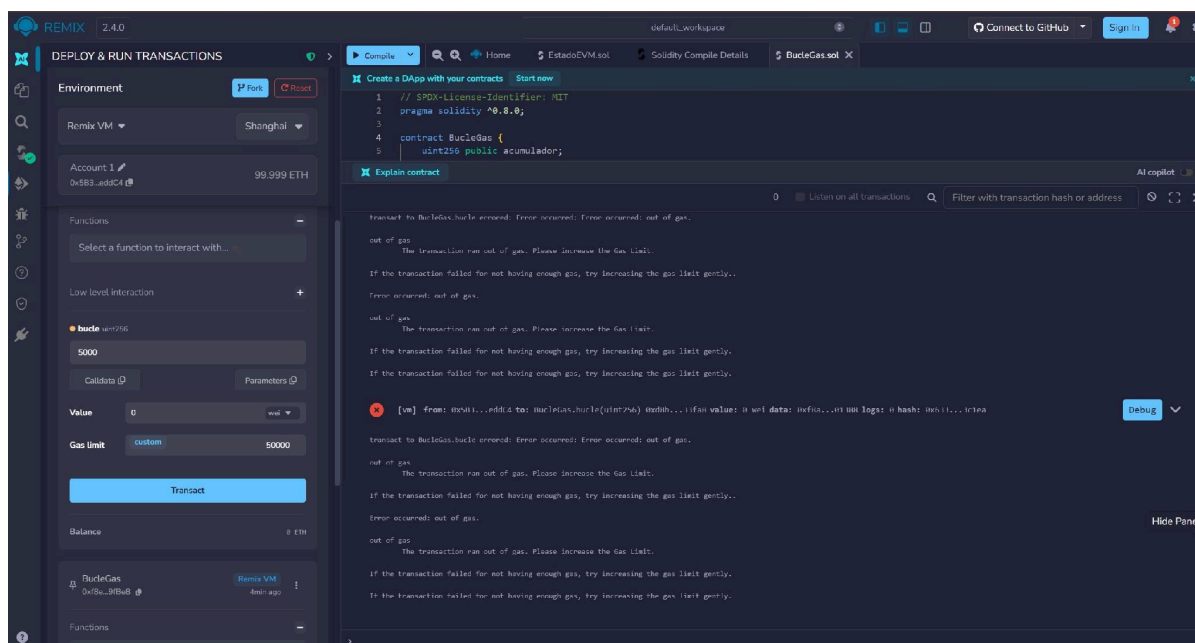
Valor de acumulador despues de la transaccion fallida: 0 (cero). Al verificar la variable acumulador mediante una llamada a la funcion getter publica, el valor retornado fue exactamente 0, confirmando que ningun cambio de estado persistio.

¿Por que Ethereum cobra gas aunque la transaccion haya fallado? Desde la perspectiva del validador: el nodo ya gasto recursos computacionales reales —ciclos de CPU, memoria, electricidad— ejecutando la transaccion hasta que se agoto el gas. Ese trabajo fisico no puede "deshacerse". Cobrar el gas remunera ese esfuerzo y, sobre todo, previene abusos: si las transacciones fallidas fueran gratuitas, un atacante podria inundar la red con computo pesado sin coste alguno, ejecutando un ataque de denegacion de servicio economicamente gratuito (Ethereum Foundation, 2023a).

¿Como resuelve el gas el Problema de la Parada de Turing? El Problema de la Parada, formulado por Alan Turing en 1936, demuestra que no existe ningun algoritmo general capaz de decidir, para cualquier programa y entrada arbitraria, si ese programa terminara o se ejecutara indefinidamente. En una blockchain publica donde miles de nodos deben ejecutar el mismo codigo, un contrato con un bucle infinito paralizaria simultaneamente toda la red.

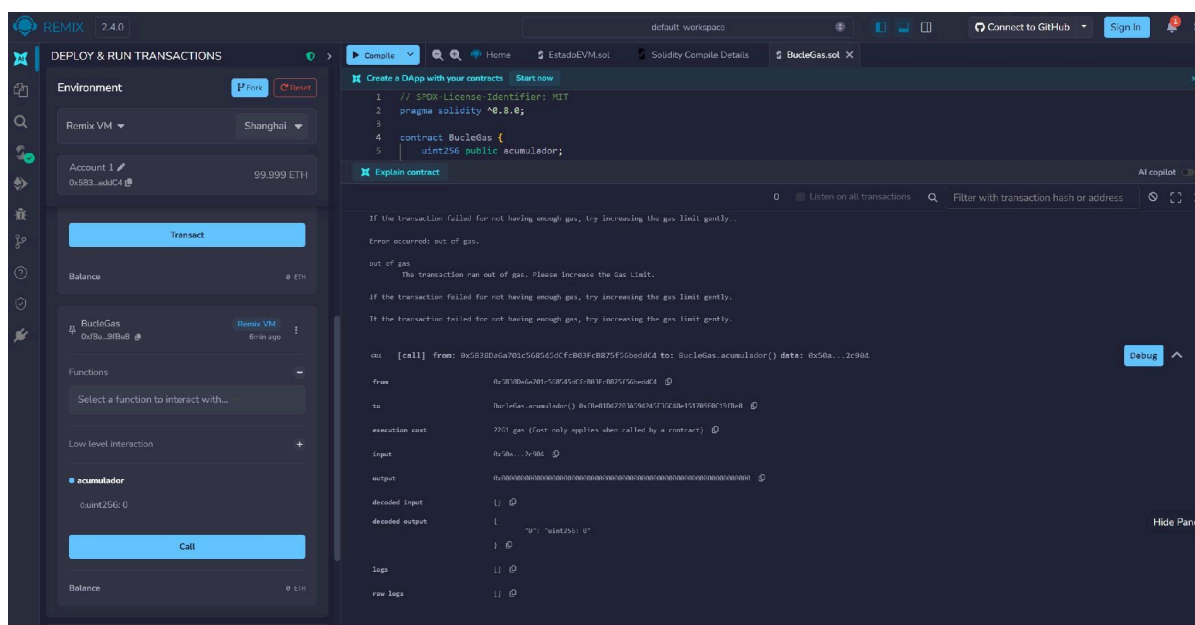
El gas es la solucion de ingenieria a este limite teorico. En lugar de intentar predecir si un programa termina (imposible segun Turing), Ethereum acota su ejecucion: cada opcode consume una cantidad determinada de gas y cada transaccion lleva un limite de gas finito. Si el computo excede ese limite, la EVM lo detiene a la fuerza y revierte. Asi, ninguna transaccion puede ejecutarse indefinidamente: la terminacion esta garantizada no por resolver el Halting Problem, sino por imponer un presupuesto que necesariamente se agota (Turing, 1936; Buterin, 2014).

Figura 10



Nota. Resultado de la ejecución de BucleGas.bucle(5000) con un Gas Limit personalizado de 50,000. La consola muestra el error "out of gas" con la marca roja de fallo, indicando que la transacción fue revertida completamente por la EVM al agotarse el presupuesto de gas.

Figura 11



Nota. Llamada de lectura (call) a la función getter acumulador() del contrato BucleGas tras el fallo Out-of-Gas. El decoded output muestra "0": uint256: 0, confirmando que el principio de atomicidad de la EVM revirtió completamente todos los cambios de estado de la transacción fallida.

Parte 5 - EIP-1559: Calculo del Costo Real de una Transaccion

Se analizo una transaccion real de tipo Contract Interaction en la red principal de Ethereum mediante Etherscan. La transaccion seleccionada corresponde a una operacion "Mint Public" del token ERC-721 IPO SPCX, registrada en el bloque 25,304,570 el 12 de junio de 2026.

Campos de Gas de la Transaccion Analizada

Campo	Valor registrado
Gas Limit	142,161
Gas Used	109,919 (77.32%)
Base Fee Per Gas	0.147760315 Gwei
Max Priority Fee Per Gas	0.03 Gwei
Max Fee Per Gas	0.18 Gwei
Transaction Fee	0.000019539236064485 ETH (\$0.03)

Verificacion Manual de la Formula EIP-1559

Aplicando la formula EIP-1559: Tarifa efectiva por gas = Base Fee + Priority Fee = $0.147760315 + 0.03 = 0.177760315$ Gwei. La tarifa total se calcula como: Tarifa total = Tarifa efectiva x Gas Used / 1,000,000,000 = $0.177760315 \times 109,919 / 1,000,000,000 = 0.00001953923...$ ETH. El resultado coincide exactamente con el campo Transaction Fee reportado por Etherscan: 0.000019539236064485 ETH, validando la formula (Ethereum EIP-1559, 2021).

Desglose del Destino del ETH Pagado

El ETH quemado corresponde a la porcion del Base Fee: $0.147760315 \text{ Gwei} \times 109,919 / 1e9 = 0.000016241666$ ETH. Etherscan confirma: Burnt: 0.000016241666064485 ETH. Esta cantidad se destruye permanentemente de la oferta circulante, implementando el mecanismo deflacionario central de EIP-1559.

El ETH recibido por el validador corresponde a la Priority Fee (propina): $0.03 \text{ Gwei} \times 109,919 / 1e9 = 0.0000032975$ ETH. El usuario adicionalmente ahorro la diferencia entre el Max Fee y la tarifa efectiva: Txn Savings = 0.000000246183935515 ETH, ya que MaxFee (0.18 Gwei) supero a la tarifa efectiva (0.177760315 Gwei).

¿Que habria ocurrido si MaxFee Per Gas = Base Fee sin margen? Si el usuario hubiera fijado MaxFee Per Gas igual al Base Fee actual sin ningun margen, y el Base Fee hubiera subido aunque fuera minimamente en el siguiente bloque, la transaccion habria quedado pendiente en el mempool hasta que el Base Fee descendiera al nivel que cubriera el MaxFee establecido, o habria expirado y sido descartada. EIP-1559 requiere que $\text{MaxFee} \geq \text{BaseFee}$ del bloque para que la transaccion sea incluida (EIP-1559, 2021).

El hard fork London, que implemento EIP-1559, se activo en el bloque numero 12,965,000 de la red principal de Ethereum, el 5 de agosto de 2021 (Ethereum Foundation, 2021).

Figura 12

Etherscan Home Blockchain Tokens NFTs Resources Developers More Sign In

Transactions </> API

TRANSACTIONS (24H)
2,085,988 (1.84%)

PENDING TRANSACTIONS (LAST 1H)
102,361 (Average)

TOTAL TRANSACTION FEE (24H)
130.39 ETH (7.62%)

AVG. TRANSACTION FEE (24H)
0.14 USD (11.87%)

More than 3,527,689,185 transactions found
(Showing the last 500k records)

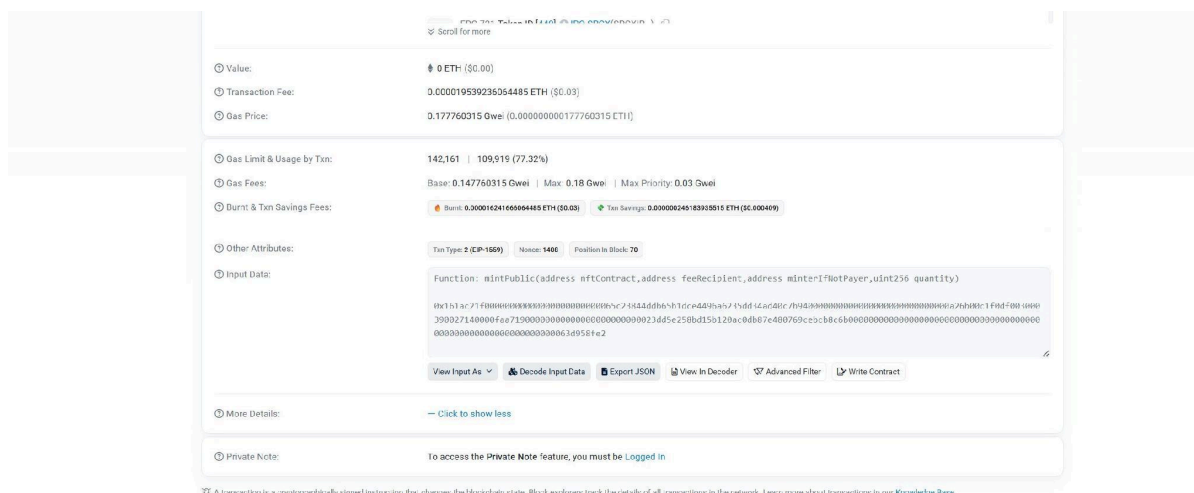
Download Page Data First < Page 7 of 10000 > Last

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0xee44c44edf9...	Transfer From	25304570	6 secs ago	Roobet 3	Circle: USDC Token	0 ETH	0.00000821
0xfe4b58aa588...	Transfer From	25304570	6 secs ago	Roobet 3	Circle: USDC Token	0 ETH	0.00000821
0x4f167be012...	Mint Public	25304570	6 secs ago	0x23dd5e25...cEbC8c68	SeaDrop	0 ETH	0.00001953
0x91ed2c5d56...	Transfer	25304570	6 secs ago	0x0d9e87bE...98Ba7a254	Circle: USDC Token	0 ETH	0.00000722
0xcabdcf30838...	Transfer	25304570	6 secs ago	Crypto.com 12	0x5eCaca02...46cFeA1E6	0.189207 ETH	0.00000377
0x61f8673b94b...	Transfer	25304570	6 secs ago	0x545EadB8...B8f04D584	Tether: USDT Stabl...	0 ETH	0.00001144
0xf61d643fba9...	Transfer	25304570	6 secs ago	0x64083D50...99e6EF331	0xA1EDa044...91753CAd1	0.078465503 ETH	0.00000382
0xfa7efdc64cd...	Transfer	25304570	6 secs ago	defi-oracle.eth	0x2BBE3c80...e9f54EE59	0 ETH	0.00001464
0xebc107d854...	Transfer From	25304570	6 secs ago	Roobet 3	Circle: USDC Token	0 ETH	0.00000878
0xaf45a3d8196...	Transfer From	25304570	6 secs ago	Roobet 3	Circle: USDC Token	0 ETH	0.00000878
0xcb5cb348f03...	Transfer From	25304570	6 secs ago	Roobet 3	Circle: USDC Token	0 ETH	0.00000878
0xc6cca2b3fd2...	Transfer From	25304570	6 secs ago	Roobet 3	Circle: USDC Token	0 ETH	0.00000878

Nota. Vista de la lista de transacciones recientes en Etherscan (etherscan.io) correspondientes al bloque 25,304,570. Se observa la transaccion seleccionada "Mint Public" junto con otras transacciones del mismo bloque, con sus respectivas tarifas de gas expresadas en ETH.

Figura 13

0x4f167be60121b39c73f774f702fc42a85f2f23daed9717d9253f5c2e243923b9 en Etherscan. Se visualizan los campos Transaction Fee (0.000019539236064485 ETH), Gas Price (0.177760315 Gwei) y la transferencia de 6 tokens ERC-721 IPO SPCX.

Figura 14

Nota. Sección de detalles de gas de la transacción en Etherscan, expandida mediante "Click to show more". Se identifican: Gas Limit (142,161), Gas Used (109,919 — 77.32%), Base Fee (0.147760315 Gwei), Max Fee (0.18 Gwei), Max Priority Fee (0.03 Gwei), ETH quemado (0.000016241666064485 ETH) y Txn Savings. El tipo de transacción es EIP-1559 (Txn Type: 2).

Parte 6 - Merkle Patricia Trie: Verifica sin Descargar la Cadena

Se ejecuto el script Python merkle.py para simular la construcción y verificación de un árbol Merkle con cuatro transacciones ficticias, demostrando como un nodo ligero puede verificar la inclusión de una transacción sin descargar el bloque completo.

Salida Completa del Script merkle.py

El script fue ejecutado desde el directorio C:\Users\diego\Documents\Codigo\Claude\Laboratorio - Arquitectura y Economía de la EVM\ mediante el comando py merkle.py en PowerShell. La salida completa obtenida fue:

```

=== MERKLE PATRICIA TRIE ===
Hojas (hashes de transacciones):
TX[0]: c581b9d87df509a875ee...
TX[1]: 52a3194df15263137d9b...
TX[2]: 166e656a76384511a742...
TX[3]: 6a9ea50a25e1e03085b6...
Raiz Merkle: 7f60ece4bb8979aaccf718a64b7e37d8f297081f6eade55383948561666a8af3
=== PRUEBA DE INCLUSION PARA TX[1] ===
¿TX[1] esta en el arbol? True
Nodos consultados para verificar: 2 de 4 totales

```

¿Cuántos nodos necesito consultar el algoritmo para verificar TX[1]? El algoritmo consulto unicamente 2 nodos de un árbol con 4 transacciones totales: el hermano izquierdo de TX[1] (TX[0]) en el Nivel 0, y el hermano derecho del nodo padre en el Nivel 1. Con esos 2 hashes mas el hash de TX[1], el algoritmo recalculo la raíz y verifíco que coincidiera con la raíz conocida.

¿Con 1,024 transacciones, cuantos nodos se necesitarian? $\log_2(1,024) = 10$ nodos. La complejidad algorítmica es $O(\log n)$: logarítmica. Duplicar el numero de transacciones solo añade un nodo mas a la prueba. Un árbol de 1,048,576 transacciones (~1 millón) solo requiere 20 nodos para verificar cualquiera de ellas (Merkle, 1980).

¿Como verifica un nodo ligero (light client) sin descargar el bloque completo? Un nodo ligero descarga unicamente los encabezados de bloque (no las transacciones completas). El encabezado contiene la raíz Merkle de las transacciones (transactionsRoot). Cuando necesita verificar una transacción: (1) solicita a un nodo completo la prueba de inclusión (los hashes hermanos del camino hasta la raíz); (2) con el hash de la transacción y los hashes de la prueba,

recalcula la raiz paso a paso; (3) compara la raiz calculada con la transactionsRoot del encabezado; (4) si coinciden, la transaccion es autentica. Todo esto sin haber descargado las otras transacciones del bloque. Esta es la base del protocolo SPV (Simplified Payment Verification) (Nakamoto, 2008).

Figura 15

```

merkle.py
1 print("=== MERKLE PATRICIA TRIE ===\n")
2 print("Hojas (hashes de transacciones):")
3 for i, tx in enumerate(transacciones):
4     print(f"TX[{i}]: {tx[:20]}...")
5
6 niveles = construir_arbol(transacciones[:])
7
8 print(f"\nNiveles del arbol: {len(niveles)}")
9 for i, nivel in enumerate(niveles):
10     etiqueta = "Raiz" if i == len(niveles) - 1 else f"Nivel {i}"
11     print(f"({etiqueta}): {[h[:12] + '...' for h in nivel]}")
12
13 raiz = niveles[-1][0]
14 print(f"\nRaiz Merkle: {raiz}")
15
16 # Genera y verifica la prueba de inclusion para Tx[1]
17 indice_a_probar = 1
18 prueba = prueba_inclusion(niveles, indice_a_probar)
19
20 print(f"\n=== PRUEBA DE INCLUSION PARA TX[{indice_a_probar}] ===")
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
25
```

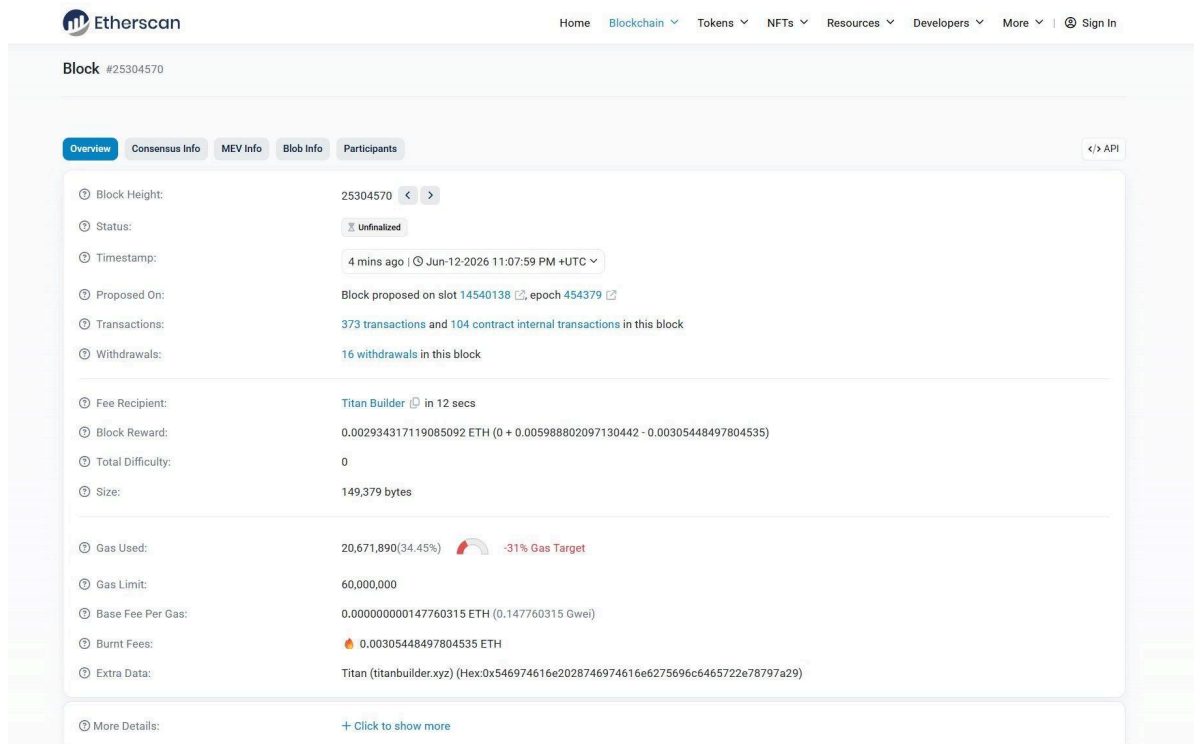
transactionsRoot como parte del objeto de bloque completo (Wood, 2022; Ethereum Foundation, 2023a).

Los Tres Tries del Encabezado de Bloque Ethereum

Trie	Campo en el header	Contiene
State Trie	stateRoot	Estado global de todas las cuentas de Ethereum (nonce, balance, storageRoot, codeHash). Representa el mundo completo despues de aplicar el bloque.
Transactions Trie	transactionsRoot	Todas las transacciones incluidas en ese bloque en orden. Permite verificar que una transaccion especifica estuvo en el bloque.
Receipts Trie	receiptsRoot	Los recibos de cada transaccion (estado exito/fallo, gas usado acumulado, logs/eventos emitidos). Permite verificar resultados sin re-ejecutar.

De los tres tries, solo el stateRoot y el withdrawalsRoot son visibles en la seccion expandida de Etherscan para el bloque 25,304,570: stateRoot = 0x0dc111e42591a800ef5d880f22bd1078cc417f65f624d2e260aaeaaeb084f318 y withdrawalsRoot = 0x965b11f9dcc268d1e383b53cced0803a99d85cb07a7a40c14d143f4746ffa63c. El transactionsRoot, que es la raiz que acredita la inclusion de las 373 transacciones del bloque, se encuentra en el mismo objeto de bloque pero Etherscan no lo despliega en la interfaz grafica principal; su recuperacion requiere una consulta RPC directa al nodo Ethereum (Ethereum Foundation, 2023a).

Figura 16



Nota. Vista general del bloque 25,304,570 en Etherscan (Jun-12-2026 11:07:59 PM UTC). Se observan los campos Block Height, Status (Unfinalized), 373 transacciones, Fee Recipient (Titan Builder), Block Reward, Gas Used (20,671,890 — 34.45%), Gas Limit (60,000,000) y Base Fee Per Gas (0.147760315 Gwei). El bloque fue propuesto en el slot 14,540,138, epoch 454,379.

Figura 17

The screenshot shows the Etherscan interface for Block #25304570. The page is titled "Block #25304570" and has tabs for Overview, Consensus Info, MEV Info, Blob Info, and Participants. The Overview tab is selected. The block is at height 25304570, status is "Unfinalized", and was proposed 4 minutes ago on Jun-12-2026 at 11:07:59 PM +UTC. It contains 373 transactions and 104 contract internal transactions, with 16 withdrawals. The fee recipient is Titan Builder, and the block reward is 0.002934317119085092 ETH. The total difficulty is 0, and the size is 149,379 bytes. Gas used is 20,671,890 (34.45%), with a gas limit of 60,000,000. The base fee per gas is 0.00000000147760315 ETH (0.147760315 Gwei). The burnt fees are 0.00305448497804535 ETH. The extra data is Titan (titanbuilder.xyz) (Hex: 0x546974616e2028746974616e6275696c6465722e78797a29). The cryptographic hashes are: Hash: 0x85862b5c199fe8233484e0f9ecf88ffa0de46fe07042cb0d025ce80c7bb99e5b, Parent Hash: 0xfa2f09c630da4b9bb8c83d9362e44fb0c9e452ebbb57cae4d91d77f2e85c3db, StateRoot: 0x0dc111e42591a800ef5d880f22bd1078cc417f65f624d2e260aaeae084f318, WithdrawalsRoot: 0x965b11f9dcc268d1e383b53cced0803a99d85cb07a7a40c14d143f4746ffa63c, and Nonce: 0x0000000000000000. A link to show more details is provided at the bottom.

Nota. Vista expandida del bloque 25,304,570 en Etherscan mostrando los campos criptograficos del encabezado disponibles en la interfaz: Hash del bloque (0x85862b5c...), Parent Hash, StateRoot (0x0dc111e4...), WithdrawalsRoot (0x965b11f9...) y Nonce. El campo transactionsRoot no aparece en esta vista de Etherscan; es recuperable mediante la llamada RPC `eth_getBlockByNumber`, que devuelve el objeto completo del bloque incluyendo los tres Merkle roots del encabezado Ethereum.

Parte 7 - Reflexion Final

Pregunta 1 - Costo Adicional de Diez Escrituras en Storage

Usando los datos de gas obtenidos en la Parte 5: Base Fee = 0.147760315 Gwei, Priority Fee = 0.03 Gwei, Tarifa efectiva = 0.177760315 Gwei por gas.

Costo de 10 operaciones SSTORE adicionales sobre slots nuevos (cero a no-cero): Gas adicional = $10 \times 20,000 = 200,000$ gas. ETH adicional = $200,000 \times 0.177760315 / 1,000,000,000 = 0.0000355520630$ ETH (aproximadamente \$0.12 USD). Si los slots ya estuvieran inicializados

(no-cero a no-cero), el costo seria: Gas adicional = $10 \times 5,000 = 50,000$ gas. ETH adicional = $50,000 \times 0.177760315 / 1,000,000,000 = 0.0000088880$ ETH (aproximadamente \$0.03 USD). El costo puede variar entre 4x y 20x segun si los slots son nuevos o ya existian, evidencia directa de por que optimizar el uso de Storage es critico en el diseno de contratos inteligentes auditados (EVM.codes, 2023).

Pregunta 2 - ¿Por que EIP-1559 hace Inviabile el Ataque de Transacciones Fallidas?

En el modelo anterior de subasta de precio (pre-EIP-1559), el atacante podia enviar transacciones con fees bajos; como las fallidas consumian gas pero el mecanismo era predecible, el costo del ataque era controlable. Con EIP-1559, el ataque se vuelve economicamente inviable por tres razones fundamentales.

Primero, la quema del Base Fee: cada transaccion, incluso las fallidas, paga y quema el Base Fee por todo el gas consumido. El ETH gastado en el ataque se destruye permanentemente, no va a ningun actor que pueda ser aliado del atacante. Segundo, el ajuste dinamico del Base Fee: si el atacante satura los bloques llevandolos al 100% de capacidad, el Base Fee sube automaticamente en un 12.5% por bloque. En pocos bloques, el costo por transaccion crece exponencialmente, creando un impuesto progresivo que el propio atacante genera. Tercero, la eliminacion de mecanismos de recuperacion: el Base Fee se quema incondicionalmente, sin posibilidad de recuperacion parcial (Ethereum EIP-1559, 2021).

La combinacion de estos tres factores hace que el ataque sea autodestructivo: cada intento aumenta el costo del siguiente y el capital gastado desaparece de la red. La relacion costo/daño es tan desfavorable para el atacante que el ataque no es economicamente sostenible.

Pregunta 3 - ¿Por que Ethereum es "Quasi-Turing Completo"?

La EVM posee el poder expresivo de una maquina de Turing: soporta bucles, condicionales, almacenamiento, recursion indirecta, y puede computar cualquier funcion computable en principio. Sin embargo, no puede ejecutar computo ilimitado: cada transaccion esta acotada por un limite de gas finito, lo que garantiza que toda ejecucion termina. Esa cota es exactamente lo que la separa de una maquina de Turing "pura" que admite tiempo y cinta infinitos. Por eso se le denomina "quasi-Turing completo": Turing-completa en expresividad, pero acotada por el gas en ejecucion real (Buterin, 2014).

La implicacion para la seguridad de la red es fundamental: esta restriccion garantiza que ningun programa puede colgar la red. En una blockchain publica donde miles de nodos deben ejecutar el mismo codigo, un contrato con un bucle infinito paralizaria todos los nodos simultaneamente — un ataque de denegacion de servicio perfecto. El modelo de gas convierte ese ataque (imposible de prevenir teoricamente por el Problema de la Parada) en uno costoso e imposible en la practica. El computo acotado hace que la red sea determinista, predecible y resistente a programas maliciosos o con errores de programacion (Turing, 1936; Wood, 2022).

Declaracion de Uso de Inteligencia Artificial

La IA (Claude Sonnet, Anthropic) fue utilizada exclusivamente como apoyo en el formato y organizacion del documento. La totalidad del desarrollo, incluyendo ejecucion de scripts, investigacion, analisis y reflexiones, fue realizada integramente por el estudiante. La IA no participo en ninguna actividad practica del laboratorio.

Referencias

- Antonopoulos, A. M., & Wood, G. (2018). Mastering Ethereum: Building smart contracts and DApps. O'Reilly Media.
- Buterin, V. (2014). Ethereum: A next-generation smart contract and decentralized application platform. <https://ethereum.org/en/whitepaper/>
- Buterin, V. (2021). Ethereum scalability, research and collaboration thread. Ethereum Research. <https://ethresear.ch/>
- Ethereum EIP-1559. (2021). EIP-1559: Fee market change for ETH 1.0 chain. Ethereum Improvement Proposals. <https://eips.ethereum.org/EIPS/eip-1559>
- Ethereum Foundation. (2021). London upgrade. <https://ethereum.org/en/history/#london>
- Ethereum Foundation. (2023a). Ethereum yellow paper appendix: Ethereum state transition function. <https://ethereum.org/en/developers/docs/evm/>
- Ethereum Foundation. (2023b). Gas and fees: EIP-2929 and EIP-3529. <https://ethereum.org/en/developers/docs/gas/>
- EVM.codes. (2023). EVM Opcodes reference. <https://www.evm.codes/>
- Merkle, R. C. (1980). Protocols for public key cryptosystems. Proceedings of the IEEE Symposium on Security and Privacy. <https://doi.org/10.1109/SP.1980.10006>
- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>
- Solidity Documentation. (2023). Solidity language specification v0.8.19. <https://docs.soliditylang.org/en/v0.8.19/>
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, 42(1), 230-265. <https://doi.org/10.1112/plms/s2-42.1.230>

Wood, G. (2022). Ethereum: A secure decentralised generalised transaction ledger (Berlin version). Ethereum Foundation. <https://ethereum.github.io/yellowpaper/paper.pdf>